



PUC Minas

Arquitetura de Computadores III

Introdução às GPUs

Origem das GPUs

- O crescimento das interfaces gráficas aumentou a demanda por processamento gráfico
- PCs e workstations passaram a dedicar parte significativa do tempo à renderização
- A Lei de Moore permitiu integrar mais lógica gráfica em hardware especializado
- Jogos eletrônicos impulsionaram fortemente a evolução das GPUs
- Consoles e PCs criaram um mercado de grande escala para aceleração gráfica

Introdução às GPUs

A Indústria de Jogos como Motor da Evolução

- O mercado de games incentivou investimentos contínuos em hardware gráfico
- GPUs evoluíram mais rapidamente que CPUs em termos de paralelismo
- O foco era maximizar taxa de renderização e processamento vetorial
- A arquitetura das GPUs evoluiu com objetivos diferentes das CPUs
- Surgiu um ecossistema próprio de hardware, APIs e linguagens

Introdução às GPUs

O Surgimento das GPUs

- Processadores gráficos ganharam identidade própria como GPUs
- GPUs foram projetadas para alta taxa de processamento paralelo
- CPUs priorizam versatilidade; GPUs priorizam throughput
- GPUs modernas possuem centenas ou milhares de unidades de ponto flutuante
- O custo relativamente baixo democratizou computação de alto desempenho

Introdução às GPUs

GPUs e Computação Geral (GPGPU)

- Pesquisadores perceberam que GPUs poderiam acelerar aplicações científicas
- Inicialmente era necessário programar usando APIs gráficas
- O processo era complexo e pouco natural para computação geral
- Surgiram linguagens inspiradas em C para programação direta da GPU
- GPUs passaram a ser usadas além de gráficos

Introdução às GPUs

CUDA e OpenCL

- CUDA foi criada pela NVIDIA para programação de GPUs em linguagem C
- OpenCL surgiu como alternativa portátil e multiplataforma
- Essas linguagens abstraem detalhes gráficos da GPU
- Permitem explorar paralelismo massivo explicitamente
- Tornaram GPUs acessíveis para computação científica

Introdução às GPUs

GPUs como Aceleradores

- GPUs não substituem CPUs; funcionam como coprocessadores
- A CPU continua executando partes sequenciais do programa
- A GPU executa regiões altamente paralelas
- GPUs podem executar algumas tarefas de forma ineficiente
- A divisão de trabalho CPU/GPU é típica em sistemas heterogêneos
- GPU é um acelerador especializado em paralelismo massivo.

Introdução às GPUs

Tamanho dos Problemas em GPUs

- GPUs normalmente trabalham com conjuntos de dados de centenas de MB até alguns GB
- CPUs geralmente possuem maior capacidade de memória principal
- GPUs modernas tipicamente possuem 4–16 GiB de memória
- Sistemas CPU podem ter dezenas ou centenas de GiB
- O volume de dados influencia diretamente a estratégia computacional

Introdução às GPUs

Latência vs. Throughput

- CPUs são otimizadas para reduzir latência
- GPUs são otimizadas para maximizar largura de banda
- CPUs usam grandes caches multinível
- GPUs usam muitas threads para esconder latência
- A estratégia arquitetural das GPUs é baseada em ocupação massiva
- GPUs escondem latência usando multithreading massivo.

Introdução às GPUs

Ocultação de Latência em GPUs

- Esse mecanismo é chamado de hardware multithreading
 1. Uma thread realiza acesso à memória
 2. O acesso possui alta latência
 3. A GPU troca rapidamente para outra thread pronta
 4. Enquanto uma thread espera memória, outras executam
 5. O processador mantém as unidades funcionais ocupadas

Introdução às GPUs

Multithreading em Hardware

- GPUs executam centenas ou milhares de threads simultaneamente
- O escalonamento é realizado por hardware
- Troca entre threads possui baixo overhead
- O objetivo é manter throughput elevado
- Estratégia diferente do uso intensivo de cache em CPUs

Introdução às GPUs

Memória em GPUs

- Memórias de GPU são otimizadas para largura de banda
- Utilizam DRAMs especializadas e barramentos largos
- O foco não é baixa latência individual
- O sistema depende de acesso paralelo massivo
- A largura de banda é crítica para alimentar milhares de threads

Introdução às GPUs

Transferência CPU ↔ GPU

- GPUs possuem memória fisicamente separada da CPU
- Dados precisam ser copiados entre CPU e GPU
- O custo de transferência pode ser significativo
- Nem toda aplicação se beneficia de aceleração em GPU
- O overhead de movimentação deve ser considerado
- A transferência de dados pode limitar o ganho de desempenho.

Introdução às GPUs

Paralelismo nas GPUs

- GPUs exploram múltiplos níveis de paralelismo
- Paralelismo SIMD dentro dos processadores
- Paralelismo MIMD entre múltiplos processadores
- Multithreading massivo para ocultar latência
- Paralelismo em nível de instrução também é explorado

Introdução às GPUs

O Modelo CUDA

- CUDA define a thread como unidade básica de programação
- Threads são agrupadas em blocos
- Blocos são distribuídos aos processadores da GPU
- O hardware organiza automaticamente a execução
- O modelo facilita escalabilidade entre GPUs diferentes

Introdução às GPUs

Organização das CUDA Threads

- Threads são executadas em grupos de 32
- Esses grupos executam em lockstep
- Todas as threads do grupo executam a mesma instrução
- O modelo lembra arquiteturas SIMD
- Divergência de controle reduz eficiência
- O grupo de 32 threads é a unidade fundamental de execução SIMD.

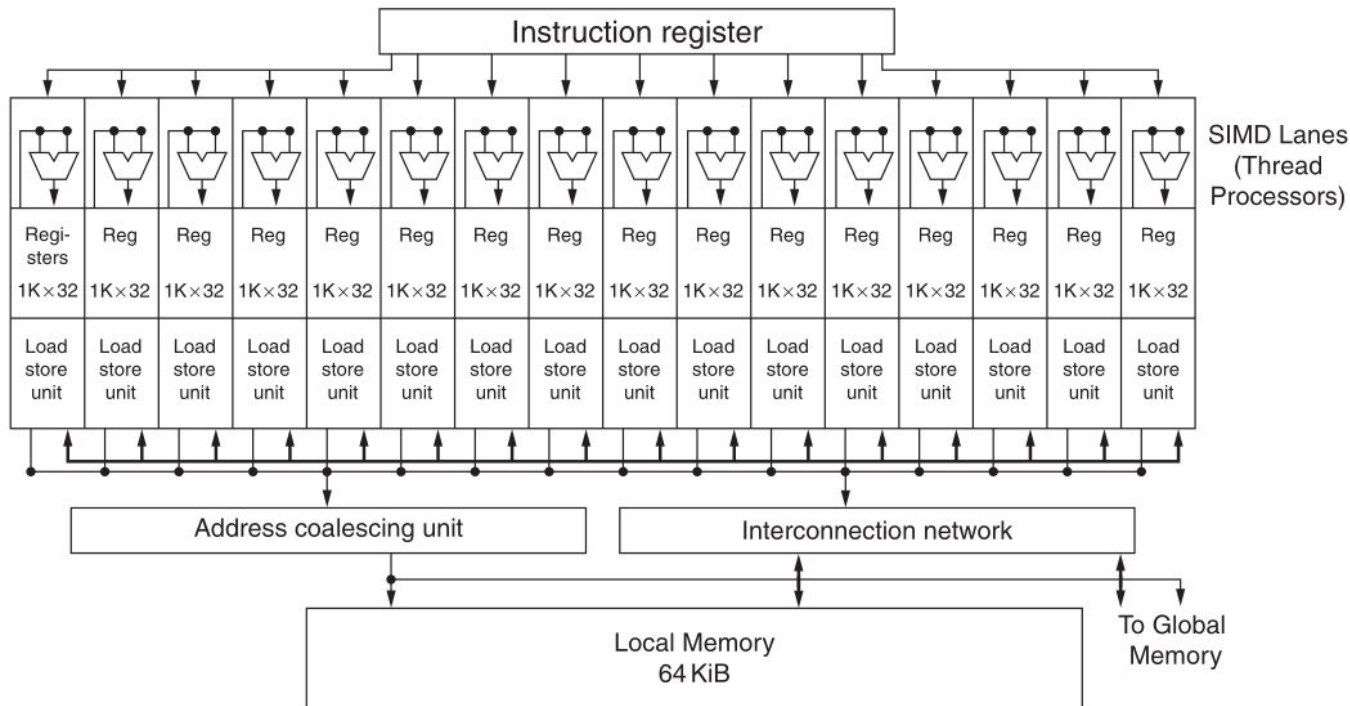
Introdução às GPUs

Arquitetura NVIDIA Fermi

- A arquitetura Fermi é representativa das GPUs modernas
- GPU composta por múltiplos processadores SIMD multithreaded
- Cada processador executa diversos grupos de threads
- Combina MIMD entre processadores e SIMD internamente
- Fermi foi projetada para computação geral e gráficos

Introdução às GPUs

Arquitetura NVIDIA Fermi



Introdução às GPUs

SIMD Processor Multithreaded

- Cada processador da GPU executa múltiplas threads SIMD
- Possui diversas unidades funcionais paralelas
- Cada thread executa instruções SIMD
- O hardware alterna rapidamente entre threads
- O modelo combina SIMD e multithreading

Introdução às GPUs

GPU como Máquina MIMD

- Uma GPU contém múltiplos processadores SIMD independentes
- Cada processador executa seus próprios blocos de threads
- O conjunto forma uma arquitetura MIMD
- Diferentes blocos podem executar tarefas independentes
- Escalabilidade depende da quantidade de processadores disponíveis

Introdução às GPUs

Escalonamento de Blocos

1. A aplicação cria blocos de threads
2. O Thread Block Scheduler distribui os blocos
3. Cada bloco é enviado a um processador SIMD
4. O hardware gerencia execução automaticamente
5. A aplicação escala entre GPUs diferentes

Introdução às GPUs

Dois Níveis de Escalonamento

- **Nível 1 — Thread Block Scheduler**
 - Distribui blocos entre processadores SIMD
- **Nível 2 — SIMD Thread Scheduler**
 - Escolhe quais threads SIMD executar
 - Alterna entre threads prontas
 - Implementa multithreading fino (fine-grained)

Introdução às GPUs

Threads SIMD

- Uma thread SIMD contém apenas instruções SIMD
- Possui contador de programa próprio
- Executa em um processador multithreaded SIMD
- O hardware controla criação e escalonamento
- Diferente de threads tradicionais POSIX

Introdução às GPUs

SIMD Lanes

- As operações SIMD são executadas em múltiplas lanes
- Cada lane executa uma parte dos dados
- As lanes funcionam de forma sincronizada
- São análogas às vector lanes de processadores vetoriais
- O paralelismo físico está nas lanes

Introdução às GPUs

Organização dos Registradores

- Cada processador possui 32.768 registradores de 32 bits
- Registradores são divididos entre as SIMD lanes
- Cada thread possui limite de até 64 registradores
- Grande quantidade de registradores suporta muitas threads
- O estado das threads fica armazenado on-chip

Introdução às GPUs

Relação entre CUDA Threads e SIMD Lanes

- Cada CUDA thread corresponde a um elemento da operação SIMD
- Uma CUDA thread executa em uma SIMD lane
- A thread SIMD completa contém 32 CUDA threads
- Cada lane processa diferentes elementos dos registradores
- O modelo lembra registradores vetoriais distribuídos

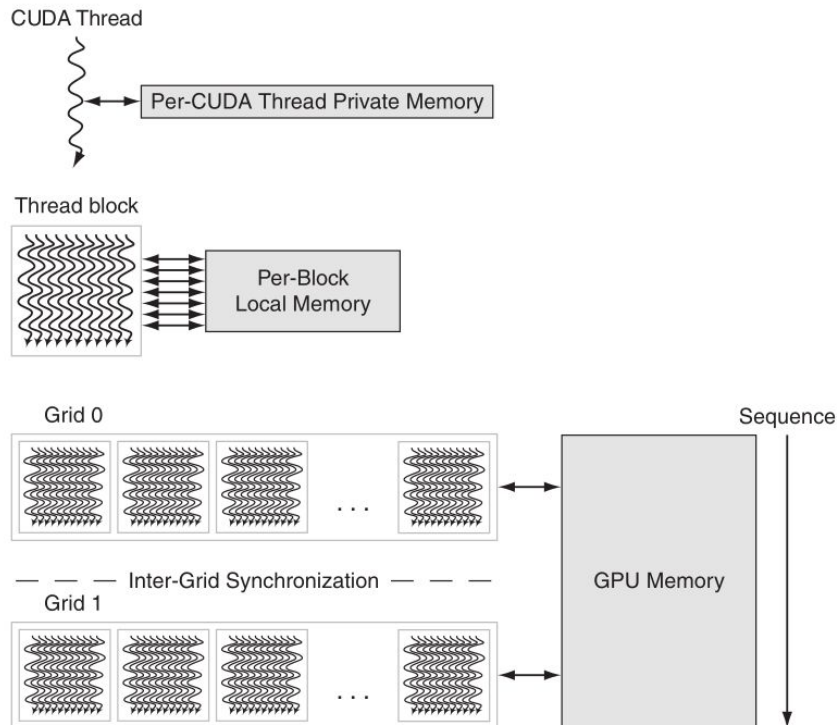
Introdução às GPUs

Estruturas de Memória da GPU

- GPUs possuem memória local on-chip
- Cada processador SIMD possui memória própria
- A GPU possui memória DRAM global compartilhada
- Memória local não é compartilhada entre processadores
- O acesso à DRAM global possui alta latência

Introdução às GPUs

Estruturas de Memória da GPU



Introdução às GPUs

Caches em GPUs

- GPUs modernas adicionaram caches mais sofisticados
- Caches ajudam em variáveis críticas à latência
- Funções e stack frames se beneficiam de caches
- Caches também reduzem consumo energético
- Ainda assim, multithreading continua central

Introdução às GPUs

GPUs vs. Arquiteturas Vetoriais

- Ambas exploram paralelismo de dados
- Ambas possuem múltiplas lanes paralelas
- GPUs utilizam multithreading intensivo
- Processadores vetoriais tradicionais dependem mais de pipelines profundos
- GPUs possuem mais registradores e mais threads simultâneas

Introdução às GPUs

GPUs vs. Processadores Vetoriais

- Ambos possuem múltiplas lanes paralelas
- Ambos exploram paralelismo de dados
- GPUs possuem forte suporte a multithreading
- Vetoriais tradicionais dependem menos de threads
- GPUs combinam MIMD + SIMD + multithreading

Introdução às GPUs

Limitações das GPUs

- GPUs não possuem paginação por demanda tradicional
- Suporte de memória virtual é limitado
- Divergência de controle reduz eficiência
- Nem todo problema possui paralelismo suficiente
- Transferência CPU/GPU pode ser gargalo

Introdução às GPUs

GPUs como Aceleradores de Domínio

- GPUs foram criadas inicialmente para gráficos
- Tornaram-se aceleradores de propósito mais amplo
- Inspiraram arquiteturas especializadas modernas
- Machine Learning ampliou ainda mais sua importância
- GPUs são exemplo clássico de aceleração por domínio

Introdução às GPUs

A terminologia de GPUs diverge da computação tradicional

Termo Descritivo	Termo Computação Geral	Termo NVIDIA (Oficial)
Proc. SIMD Multithreaded	Core (Núcleo)	Streaming Multiprocessor (SM)
Lane SIMD	Unidade Aritmética / Pipeline	CUDA Core (ou SP)
Thread de Inst. SIMD	Vetor de Instrução / SMT	Warp
Escalonador de Bloco	Distribuidor de Tarefas	Giga Thread Engine

Introdução às GPUs

Similaridades e diferenças entre Multicore com Multimedia SIMD extensions e GPUs recentes:

Feature	Multicore with SIMD	GPU
SIMD processors	8 to 32	15 to 128
SIMD lanes/processor	2 to 4	8 to 16
Multithreading hardware support for SIMD threads	2 to 4	16 to 32
Largest cache size	48 MiB	6 MiB
Size of memory address	64-bit	64-bit
Size of main memory	64 GiB to 1024 GiB	4 GiB to 16 GiB
Memory protection at level of page	Yes	Yes
Demand paging	Yes	No
Cache coherent	Yes	No

Bibliografia

Capítulo 6 - Computer Organization and Design RISC-V Edition: The Hardware Software Interface, David A. Patterson, John L. Hennessy. Editora Morgan Kaufmann, 2ª Edição, 2020.